Mini Project (DL)

In [20]:
```
# Name: Atharv Santosh Danave
# Roll No.: 11
# Project Title: Human Face Recognition
```

In [23]:
```
# Install Kaggle API
!pip install -q kaggle

# Imports
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator

import numpy as np
import pandas as pd
import cv2
```

In [24]:
```
# Upload kaggle.json file
from google.colab import files
files.upload()
```

Choose Files | No file chosen — Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving kaggle.json to kaggle.json
```

Out[24]:
```
{'kaggle.json': b'{"username":"atharv1104","key":"b1c98130ce38f11ffd1d9d5fc9a6efe0"}'}
```

In [25]:
```
# Configure Kaggle
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# Download + unzip CelebA dataset
!kaggle datasets download -d jessicali9530/celeba-dataset
!unzip -q celeba-dataset.zip
```

```
Dataset URL: https://www.kaggle.com/datasets/jessicali9530/celeba-dataset
License(s): other
Downloading celeba-dataset.zip to /content
100% 1.33G/1.33G [00:09<00:00, 155MB/s]
```

In [28]:
```
# Constants
img_height = 128
img_width = 128
batch_size = 32
epochs = 10

# Load CSV
df = pd.read_csv('list_attr_celeba.csv')

# Shuffle dataset
df = df.sample(frac=1).reset_index(drop=True)
```

```
# Convert labels (-1 → 0)
df['Smiling'] = df['Smiling'].apply(lambda x: '1' if x == 1 else '0')

# OPTIONAL: reduce dataset size for faster training
df = df.sample(20000).reset_index(drop=True)

# Split dataset
train_end = int(len(df) * 0.8)
val_end = int(len(df) * 0.9)

df_train = df[:train_end]
df_val = df[train_end:val_end]
df_test = df[val_end:]
```

In [29]:
```
# Normalize images
train_datagen = ImageDataGenerator(rescale=1./255)
val_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

# Generators
train_generator = train_datagen.flow_from_dataframe(
    dataframe=df_train,
    directory='img_align_celeba/img_align_celeba',
    x_col='image_id',
    y_col='Smiling',
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='binary'
)

val_generator = val_datagen.flow_from_dataframe(
    dataframe=df_val,
    directory='img_align_celeba/img_align_celeba',
    x_col='image_id',
    y_col='Smiling',
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='binary'
)

test_generator = test_datagen.flow_from_dataframe(
    dataframe=df_test,
    directory='img_align_celeba/img_align_celeba',
    x_col='image_id',
    y_col='Smiling',
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='binary'
)
```

```
Found 16000 validated image filenames belonging to 2 classes.
Found 2000 validated image filenames belonging to 2 classes.
Found 2000 validated image filenames belonging to 2 classes.
```

In [30]:
```
# CNN model
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(img_height, img_width, 3)),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
```

```
    MaxPooling2D(2,2),

    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),
    Dropout(0.5),

    Dense(512, activation='relu'),
    Dense(1, activation='sigmoid')  # binary output
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarn
ing: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [31]:
```
# Compile
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_4 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_5 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_1 (Flatten)	(None, 25088)	0
dropout_1 (Dropout)	(None, 25088)	0
dense_2 (Dense)	(None, 512)	12,845,568
dense_3 (Dense)	(None, 1)	513

Total params: 12,939,329 (49.36 MB)

Trainable params: 12,939,329 (49.36 MB)

Non-trainable params: 0 (0.00 B)

In [32]:
```
# Train
history = model.fit(
    train_generator,
    epochs=epochs,
```

```
    validation_data=val_generator
)
```

```
Epoch 1/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 26s 39ms/step - accuracy: 0.7749 - loss: 0.4520 - val_accuracy: 0.88
45 - val_loss: 0.2653
Epoch 2/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 34s 35ms/step - accuracy: 0.8784 - loss: 0.2933 - val_accuracy: 0.91
45 - val_loss: 0.2314
Epoch 3/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 16s 31ms/step - accuracy: 0.8913 - loss: 0.2598 - val_accuracy: 0.90
50 - val_loss: 0.2278
Epoch 4/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 16s 32ms/step - accuracy: 0.8969 - loss: 0.2455 - val_accuracy: 0.91
00 - val_loss: 0.2175
Epoch 5/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 16s 32ms/step - accuracy: 0.9038 - loss: 0.2326 - val_accuracy: 0.91
35 - val_loss: 0.2018
Epoch 6/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 17s 34ms/step - accuracy: 0.9049 - loss: 0.2233 - val_accuracy: 0.91
55 - val_loss: 0.2060
Epoch 7/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 16s 31ms/step - accuracy: 0.9112 - loss: 0.2108 - val_accuracy: 0.91
00 - val_loss: 0.2087
Epoch 8/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 16s 32ms/step - accuracy: 0.9160 - loss: 0.1989 - val_accuracy: 0.91
85 - val_loss: 0.1987
Epoch 9/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 16s 32ms/step - accuracy: 0.9177 - loss: 0.1910 - val_accuracy: 0.90
50 - val_loss: 0.2359
Epoch 10/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 17s 34ms/step - accuracy: 0.9226 - loss: 0.1824 - val_accuracy: 0.91
50 - val_loss: 0.1980
```

In [33]:
```
# Test performance
loss, accuracy = model.evaluate(test_generator)
print("Test accuracy:", accuracy)
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 2s 30ms/step - accuracy: 0.9095 - loss: 0.2425
Test accuracy: 0.909500002861023
```

In [34]:
```
# Load sample image
img = cv2.imread('img_align_celeba/img_align_celeba/000001.jpg')
img = cv2.resize(img, (img_height, img_width))
img = img / 255.0
img = np.expand_dims(img, axis=0)

# Predict
prediction = model.predict(img)

print("Smiling Probability:", prediction[0][0])
print("Smiling" if prediction[0][0] > 0.5 else "Not Smiling")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
Smiling Probability: 0.7455114
Smiling
```

In [ ]:
```

```